

Link for HTML extension setup in Visual studio

<https://www.bing.com/videos/riverview/relatedvideo?q=how+to+install+html+extension+in+visual+studio&mid=B16C8F1948EF4F37281EB16C8F1948EF4F37281E>

Git is a distributed version control system that allows developers to track changes in their codebase, collaborate with others, and manage different versions of their software projects. Here are some fundamental concepts and common Git commands to get you started with using Git:

1. **Initializing a Git Repository:** To start using Git in a project, you need to initialize a Git repository. Navigate to your project's root directory in the command line and run:

```
code  
git init
```

2. **Cloning a Repository:** To work on an existing Git repository, you can clone it using the `git clone` command:

```
code  
git clone <repository URL>
```

3. **Checking the Status:** To see which files are staged or modified in your project, use the following command:

```
code  
git status
```

4. **Adding Changes:** To stage changes for the next commit, use the `git add` command:

```
code  
git add <file(s)>
```

5. **Committing Changes:** After staging your changes, commit them with a meaningful message:

```
code  
git commit -m "Your commit message here"
```

6. **Viewing Commit History:** To see the commit history, use the `git log` command:

```
code  
git log
```

7. **Creating Branches:** Git allows you to work on different branches to isolate features or fixes. To create a new branch, use:

code

```
git branch <branch-name>
```

8. **Switching Branches:** To switch to a different branch, use the `git checkout` command:

code

```
git checkout <branch-name>
```

9. **Merging Branches:** To merge changes from one branch into another, use:

phpCopy code

```
git merge <branch-name>
```

10. **Pulling and Pushing Changes:** To update your local repository with changes from a remote repository, use:

Copy code

```
git pull
```

To push your local changes to a remote repository, use:

code

```
git push
```

11. **Handling Conflicts:** Conflicts can occur when merging branches. Git will mark these conflicts in the affected files, and you must resolve them manually.

12. **Ignoring Files:** You can create a `.gitignore` file in your repository to specify which files or directories should be ignored by Git.

13. **Undoing Changes:** To discard changes in your working directory, use `git checkout` with the file name or commit hash. Be cautious as this action is not reversible.

14. **Stashing Changes:** If you have uncommitted changes but need to switch branches, you can stash your changes temporarily using `git stash`.

15. Reverting Commits: To undo a commit and create a new commit that undoes the changes introduced by the previous commit, use `git revert`.

These are some of the basic Git commands and concepts. Git offers many more advanced features and workflows for version control and collaboration, but these should help you get started with using Git for your software development projects.

Practical Sample

Here's a practical sample of how to use Git for version control in a typical software development scenario. In this example, we'll go through the steps to create a new Git repository, make changes to files, commit those changes, and push them to a remote repository.

Step 1: Set Up Git

If you haven't already, you need to install Git on your computer and configure it with your name and email address. You can do this using the following commands in your terminal:

```
bashCopy code
git config --global user.name "Your Name" git config --global user.email "youremail@example.com"
```

Step 2: Create a New Git Repository

Let's create a new directory for our project and initialize a Git repository in it:

```
code
mkdir my_project
cd my_project
git init
```

Step 3: Create and Edit Files

Now, let's create some files and make some changes:

```
code
touch index.html touch styles.css
```

Edit these files with your preferred text editor or IDE to add some content.

Step 4: Check the Status of Your Repository

You can check the status of your Git repository to see which files have been modified and need to be committed:

```
code
git status
```

Step 5: Add Changes to the Staging Area

To start tracking changes, you need to add the files to the staging area:

code

```
git add index.html styles.css
```

Step 6: Commit Changes

Commit your changes with a meaningful commit message:

code

```
git commit -m "Initial commit. Added index.html and styles.css"
```

Step 7: Create a Remote Repository (Optional)

If you want to push your changes to a remote repository (e.g., GitHub or GitLab), create a repository there and obtain the repository URL.

Step 8: Connect to a Remote Repository

If you have a remote repository, connect your local repository to it:

code

```
git remote add origin <repository_url>
```

Step 9: Push Changes to the Remote Repository

Now, push your local changes to the remote repository:

```
git push -u origin main
```

This command pushes your changes to the "main" branch (you can replace "main" with "master" or another branch name if needed).

Step 10: Pull Changes from the Remote Repository

If you're working with a team, you'll often need to pull changes made by others into your local repository:

code

```
git pull origin main
```

These are the basic steps involved in using Git for version control. You'll continue to make changes, commit them, and push them to the remote repository as your project progresses. Additionally, you'll collaborate with others by pulling and pushing changes to maintain a synchronized codebase.